

Atividade – Identificação de Componentes e Classes do Sistema

Disciplina: SSC0124 — Análise e Projeto Orientados a Objetos (APOO)

Docente: Profa. Dra. Lina Garcés

Aluno: Rafael Feltrim

Nº USP: 15942812

Atividade: Plataforma de Venda de Passagens Rodoviárias Integrada

1. Identificação dos Componentes Explícitos do Sistema

Aplicando os princípios de **Alta Coesão** e **Responsabilidade Única (SRP)**, os Casos de Uso que manipulam e processam informações de domínio afins foram agrupados nos seguintes componentes explícitos:

- 1. Componente de Catálogo e Viagens (ComponenteCatalogo):**
 - a. *Responsabilidade:* Centralizar a gestão de ofertas de transporte, abrangendo rotas, linhas, horários, preços base, cidades atendidas e ônibus disponibilizados pelas empresas parceiras.
- 2. Componente de Vendas e Passagens (ComponenteVendas):**
 - a. *Responsabilidade:* Orquestrar todo o ciclo de checkout da compra, seleção de poltronas, reserva temporária (bloqueio com timeout concorrente), faturamento e emissão do bilhete com integração de mensageria.
- 3. Componente de Pós-Venda e Atendimento (ComponentePosVenda):**
 - a. *Responsabilidade:* Gerenciar as ocorrências e alterações posteriores à emissão dos bilhetes, incluindo cancelamento, remarcação de passagens, apuração de multas e solicitação de estornos.
- 4. Componente Operacional e Embarque (ComponenteOperacional):**
 - a. *Responsabilidade:* Fornecer dados operacionais em tempo real para a equipe de bordo (motoristas e despachantes), tais como escalas de viagem e lista de passageiros embarcados.
- 5. Componente de Gestão e Auditoria (ComponenteGestao):**

- a. *Responsabilidade*: Consolidar métricas operacionais, volume de vendas, faturamento geral e relatórios analíticos para os administradores da plataforma.

2. Casos de Uso sob Responsabilidade de Cada Componente

Componente Explícito	Casos de Uso sob sua Responsabilidade	Descrição do Domínio
ComponenteCatalogo	• Pesquisar Viagens • Comparar Preços e Horários • Manter Catálogo de Viagens (Rotas, Horários, Preços, Ônibus)	Consulta e manutenção da malha viária e disponibilidade de horários.
ComponenteVendas	• Comprar Passagem • Selecionar Assentos • Realizar Reserva Temporária • Efetuar Pagamento Online • Notificar Usuário	Concretização comercial da venda, reserva de assentos e cobrança.
ComponentePosVenda	• Gerenciar Passagem (Base) • Cancelar Passagem • Alterar Passagem • Solicitar Reembolso • Gerenciar Cancelamentos e Reembolsos • Efetuar Reembolso	Atendimento pós-compra, políticas de remarcação e estornos.
ComponenteOperacional	• Consultar Escala de Viagens • Consultar Lista de Passageiros • Acompanhar Vendas das Viagens	Acompanhamento logístico e apoio ao motorista/empresa.
ComponenteGestao	• Monitorar Operações do Sistema • Gerar Relatórios	Governança corporativa,

	(Base)• Gerar Relatório Financeiro• Gerar Relatório Operacional	auditoria e inteligência de negócios.
--	---	---------------------------------------

3. Identificação dos Componentes Implícitos do Sistema

Os componentes implícitos são subsistemas técnicos e de infraestrutura indispensáveis para atender aos requisitos de desempenho, concorrência, segurança e integração externa, que não aparecem diretamente como bolhas no diagrama funcional de Casos de Uso:

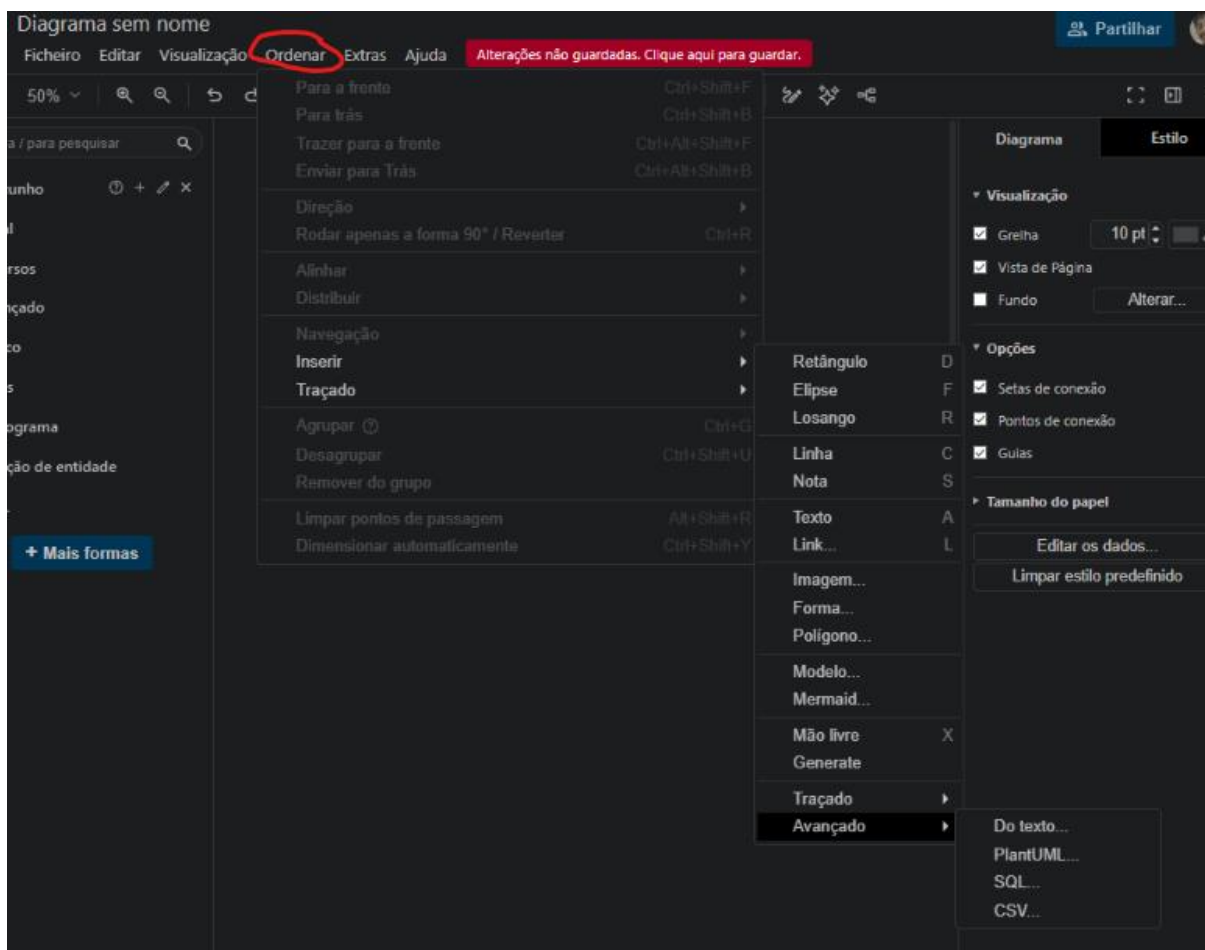
1. **DB_Catalogo (Banco de Dados de Catálogo):**
 - a. Repositório otimizado para consultas rápidas de trechos, itinerários, frotas de ônibus e tabelas tarifárias.
2. **DB_Transacional_Vendas (Banco de Dados de Vendas e Reservas):**
 - a. Repositório com suporte a transações ACID estritas para controle de concorrência em poltronas e garantia de não sobreposição de compras (*double booking*).
3. **DB_Auditoria_Analytics (Repositório de Dados Operacionais):**
 - a. Armazenamento de eventos para relatórios gerenciais e histórico de logs.
4. **GatewayPagamento (Módulo Externo de Cobrança):**
 - a. API/SDK externa (ex.: Cielo, Stripe, Adyen, PIX) responsável pela autorização de cartões e liquidação financeira.
5. **ServicoNotificacao (Módulo Externo de Mensageria):**
 - a. Provedores de comunicação multicanal (ex.: Twilio WhatsApp API, AWS SES / SendGrid) para envio de bilhetes e alertas de atraso.
6. **MessageBroker_Notificacoes (Fila/Barramento de Mensagens Assíncronas):**
 - a. Middleware de enfileiramento (ex.: RabbitMQ, AWS SQS) que desacopla o checkout da compra do disparo de e-mails/mensagens, garantindo resiliência.
7. **ComponenteAutenticacao_Seguranca:**
 - a. Mecanismo centralizado de autenticação, RBAC (Role-Based Access Control) e geração de tokens JWT para passageiros, viagens, motoristas e administradores.

4. Diagrama de Componentes UML seguindo o Padrão Arquitetural MVC

Conforme as diretrizes da Aula 09 (Slides 24 e 28), o sistema é estruturado em camadas no padrão **MVC**:

- **View (<<view>>):**
 - Interfaces dedicadas para cada perfil de usuário.
- **Controller (<<controller>>):**
 - Orquestradores que recebem requisições da View, aplicam regras de negócio e acionam o Model.
- **Model (<<model>>):**
 - Componentes especialistas do domínio do problema.
- **Data / Services (<<data>> / <<service>>):**
 - Componentes de acesso a dados persistentes e integrações com serviços terceiros.

Código PlantUML (Para importar, localizar “Ordenar”> Inserir > Avançado > PlantUML no diagrams.net):



Pós inserir o código, basta editar e mover itens pré gerados seguindo padrão MVC.

```
@startuml
```

```
left to right direction
```

```
skinparam componentStyle uml2
```

```
skinparam packageStyle rectangle
```

```
actor Passageiro
```

```
actor "Empresa de Viação" as Viacao
```

```
actor Motorista
```

```
actor Administrador
```

```
package "Camada View (<<view>>)" {
    [ViewPassageiro\n(Web & Mobile)] as VP
    [ViewViacao\n(Portal Parceiro)] as VV
    [ViewMotorista\n(App Operacional)] as VM
    [ViewAdmin\n(Dashboard)] as VA
}
```

```
}
```

```
package "Camada Controller (<<controller>>)" {  
    [ControllerCatalogo] as CC  
    [ControllerVendas] as CV  
    [ControllerPosVenda] as CPV  
    [ControllerOperacional] as CO  
    [ControllerGestao] as CG  
}
```

```
package "Camada Model (<<model>>)" {  
    [ComponenteCatalogo] as MC  
    [ComponenteVendas] as MV  
    [ComponentePosVenda] as MPV  
    [ComponenteOperacional] as MO  
    [ComponenteGestao] as MG  
}
```

```
package "Camada de Dados & Servios (<<data>> / <<external>>)" {  
    [ComponenteAcessoDados_Catalogo] as DAC  
    [ComponenteAcessoDados_Vendas] as DAV  
    database "DB_Catalogo" as DBC  
    database "DB_Vendas" as DBV  
    [GatewayPagamento\n<<external>>] as GP  
    [ServicoNotificacao\n<<external>>] as SN  
}
```

' Interaes Usurios -> Views

Passageiro --> VP

Viacao --> VV

Motorista --> VM

Administrador --> VA

' Views -> Controllers (<<use>>)

VP ..> CC : <<use>>

VP ..> CV : <<use>>

VP ..> CPV : <<use>>

VV ..> CC : <<use>>

VV ..> CPV : <<use>>

VM ..> CO : <<use>>

VA ..> CG : <<use>>

' Controllers -> Models (<<use>>)

CC ..> MC : <<use>>

CV ..> MV : <<use>>

CPV ..> MPV : <<use>>

CO ..> MO : <<use>>

CG ..> MG : <<use>>

' Colaborações entre Models (Inter-componentes)

MV ..> MC : <<use>> (dados de viagem)

MPV ..> MV : <<use>> (estorno de bilhete)

MO ..> MV : <<use>> (lista de passageiros)

MG ..> MV : <<use>> (totais de vendas)

' Models -> Acesso a Dados & Serviços Externos

MC ..> DAC : <<use>>

DAC ..> DBC : <<use>>

MV ..> DAV : <<use>>

DAV ..> DBV : <<use>>

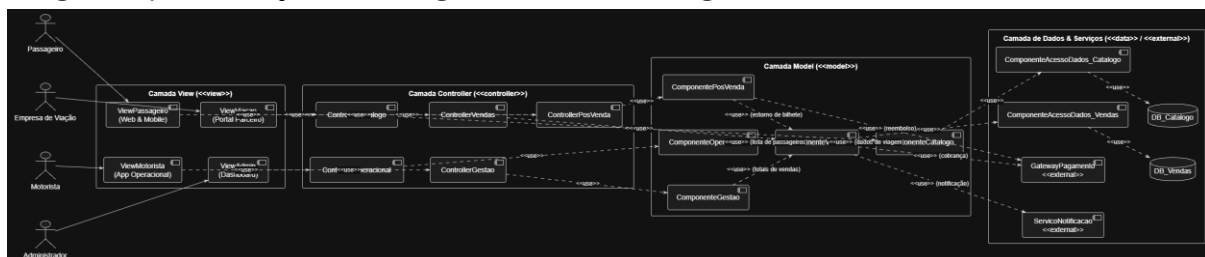
MV ..> GP : <<use>> (cobrança)

MV ..> SN : <<use>> (notificação)

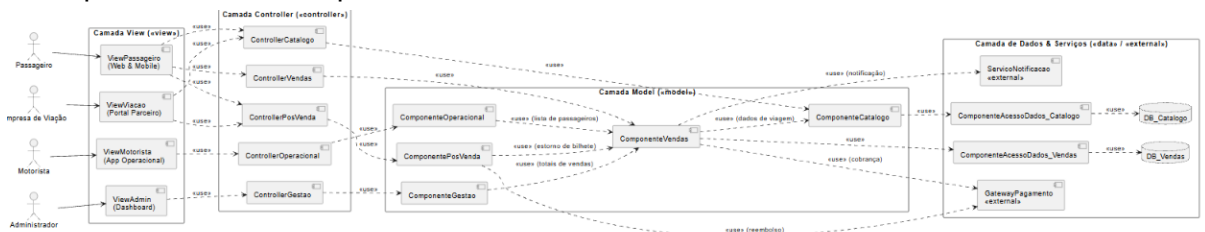
MPV ..> GP : <<use>> (reembolso)

@endum1

Diagrama pós inserção do código PlantUML no diagrams:



Pós aplicar o Padrão Arquitetural MVC com fundo branco:



5. Seleção de Componente, Identificação de Classes de Domínio e Atributos

Componente Selecionado:

- **ComponenteVendas (Vendas e Passagens) *Justificativa:***
 - É o núcleo de valor do negócio, responsável pelo checkout, concorrência de poltronas, transações financeiras e emissão de bilhetes.

a. Identificação das Classes por Categoria de Domínio (Diretrizes da Aula 09):

- **Coisas Tangíveis / Físicas:**
 - Onibus, Poltrona
- **Perfis de Usuários / Atores de Domínio:**
 - Passageiro, EmpresaViacao
- **Transações Comerciais:**
 - CompraPassagem, Pagamento, ReservaPoltrona
- **Conceitos Canônicos / Entidades do Domínio:**
 - BilhetePassagem, Viagem, Trecho
- **Comunicações / Eventos:**
 - NotificacaoEmbarque

b. Definição de Atributos Tipados das Classes (Sem Métodos)

1. Passageiro

- a. idPassageiro: Integer
- b. nomeCompleto: String
- c. documentoCpf: String
- d. email: String
- e. telefone: String

2. EmpresaViacao

- a. idViacao: Integer
- b. razaoSocial: String
- c. cnpj: String
- d. telefoneContato: String

3. Viagem

- a. idViagem: Integer
- b. codigoIdentificador: String
- c. dataPartida: Date
- d. horarioPartida: Time

- e. horarioChegadaPrevisto: Time
- f. precoBase: Decimal
- g. statusViagem: String

4. Trecho

- a. idTrecho: Integer
- b. cidadeOrigem: String
- c. cidadeDestino: String
- d. terminalOrigem: String
- e. terminalDestino: String
- f. distanciaKm: Integer

5. Onibus

- a. idOnibus: Integer
- b. placaVeiculo: String
- c. categoriaServico: String (ex: Convencional, Executivo, Leito)
- d. capacidadeTotalAssentos: Integer

6. Poltrona

- a. idPoltrona: Integer
- b. numeroAssento: Integer
- c. localizacao: String (ex: Janela, Corredor)
- d. piso: Integer (ex: 1 ou 2)

7. ReservaPoltrona

- a. idReserva: Integer
- b. dataHoraInicioReserva: DateTime
- c. dataHoraExpiracaoTimeout: DateTime
- d. statusReserva: String (ex: Pendente, Confirmada, Expirada)

8. CompraPassagem

- a. idCompra: Integer
- b. codigoLocalizador: String
- c. dataHoraCompra: DateTime
- d. valorTotalBruto: Decimal
- e. valorDescontoCupom: Decimal
- f. valorTotalLiquido: Decimal
- g. statusCompra: String (ex: Processando, Paga, Cancelada)

9. BilhetePassagem

- a. numeroBilheteEletronico: String
- b. codigoQrCodeEmbarque: String
- c. plataformaEmbarque: String
- d. nomePassageiroImpresso: String
- e. documentoPassageiroImpresso: String
- f. tarifaFinal: Decimal

10. Pagamento

- a. idPagamento: Integer

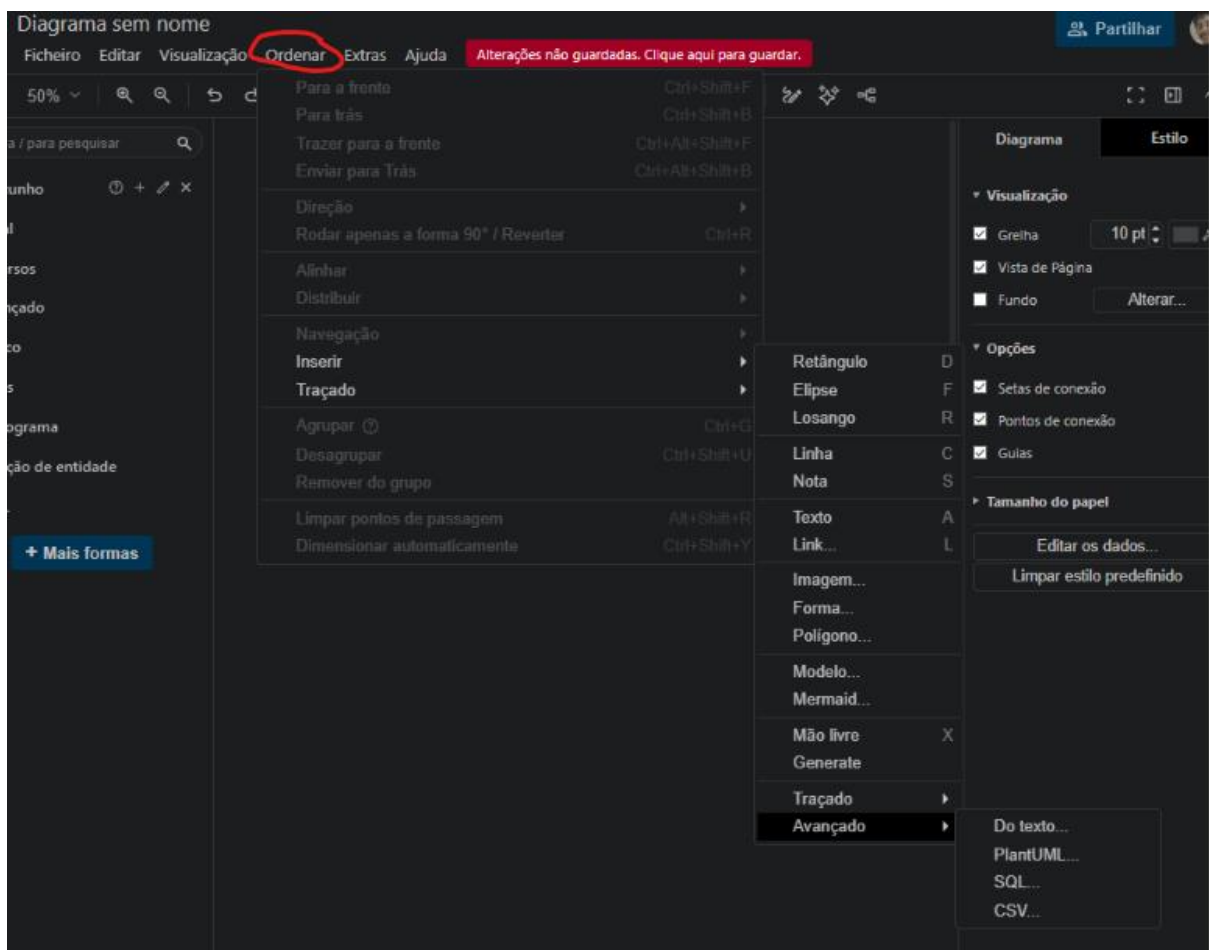
- b. `metodoPagamento`: `String` (ex: `CartaoCredito`, `Pix`, `Boleto`)
 - c. `valorPago`: `Decimal`
 - d. `dataHoraProcessamento`: `DateTime`
 - e. `codigoAutorizacaoGateway`: `String`
 - f. `statusTransacao`: `String` (ex: `Aprovado`, `Recusado`, `Estornado`)
- 11. NotificacaoEmbarque**
- a. `idNotificacao`: `Integer`
 - b. `canalEnvio`: `String` (ex: `Email`, `WhatsApp`)
 - c. `destinatarioContato`: `String`
 - d. `dataHoraEnvio`: `DateTime`
 - e. `statusEntrega`: `String` (ex: `Enviada`, `Entregue`, `Falha`)

6. Diagrama de Modelo Conceitual em UML (Classes de Domínio e Relacionamentos)

O diagrama abaixo contempla:

- **Associações** com multiplicidades rigorosamente definidas (1, 1..*, 0..1, etc.).
- **Composição** (◆ / vínculo forte):
 - `CompraPassagem` é composta por 1..* `BilhetePassagem` (os bilhetes emitidos pertencem exclusivamente àquela compra e não têm ciclo de vida autônomo sem ela).
 - `Onibus` é composto por 1..* `Poltrona` (as poltronas fazem parte física da estrutura do veículo).
- **Agregação** (◇ / vínculo fraco):
 - `CompraPassagem` agrega 1 `Pagamento` (o comprovante da transação financeira é uma entidade que complementa a compra, mas possui registro fiscal independente).
 - `Viagem` agrega 1 `Onibus` (o ônibus cumpre a escala da viagem, mas existe e opera em múltiplas outras viagens).
- **Herança / Especialização**: aplicada em categorias de assentos/veículos e tipos de transação.

Código PlantUML (Importar em "Ordenar"> Inserir > Avançado > PlantUML no diagrams.net):



@startuml

skinparam classAttributeIconSize 0

skinparam linetype ortho

```
class Passageiro {
    +idPassageiro: Integer
    +nomeCompleto: String
    +documentoCpf: String
    +email: String
    +telefone: String
}
```

```
class EmpresaViacao {
    +idViacao: Integer
    +razaoSocial: String
    +cnpj: String
    +telefoneContato: String
}
```

```
}
```

```
class Trecho {  
    +idTrecho: Integer  
    +cidadeOrigem: String  
    +cidadeDestino: String  
    +terminalOrigem: String  
    +terminalDestino: String  
    +distanciaKm: Integer  
}
```

```
class Viagem {  
    +idViagem: Integer  
    +codigoIdentificador: String  
    +dataPartida: Date  
    +horarioPartida: Time  
    +horarioChegadaPrevisto: Time  
    +precoBase: Decimal  
    +statusViagem: String  
}
```

```
class Onibus {  
    +idOnibus: Integer  
    +placaVeiculo: String  
    +categoriaServico: String  
    +capacidadeTotalAssentos: Integer  
}
```

```
class Poltrona {  
    +idPoltrona: Integer  
    +numeroAssento: Integer  
    +localizacao: String  
    +pisos: Integer  
}
```

```
class ReservaPoltrona {  
    +idReserva: Integer  
    +dataHoraInicioReserva: DateTime  
    +dataHoraExpiracaoTimeout: DateTime  
    +statusReserva: String  
}
```

```

class CompraPassagem {
    +idCompra: Integer
    +codigoLocalizador: String
    +dataHoraCompra: DateTime
    +valorTotalBruto: Decimal
    +valorDescontoCupom: Decimal
    +valorTotalLiquido: Decimal
    +statusCompra: String
}

```

```

class BilhetePassagem {
    +numeroBilheteEletronico: String
    +codigoQrCodeEmbarque: String
    +plataformaEmbarque: String
    +nomePassageiroImpresso: String
    +documentoPassageiroImpresso: String
    +tarifaFinal: Decimal
}

```

```

class Pagamento {
    +idPagamento: Integer
    +metodoPagamento: String
    +valorPago: Decimal
    +dataHoraProcessamento: DateTime
    +codigoAutorizacaoGateway: String
    +statusTransacao: String
}

```

```

class NotificacaoEmbarque {
    +idNotificacao: Integer
    +canalEnvio: String
    +destinatarioContato: String
    +dataHoraEnvio: DateTime
    +statusEntrega: String
}

```

' Relacionamentos e Associações

' Viacao opera Viagens

EmpresaViacao "1" -- "0..*" Viagem : opera >

' Viagem atende um Trecho

Viagem "0..*" -- "1" Trecho : percorre >

' Viagem agrega um Onibus (Agregação: o ônibus continua existindo sem a viagem)

Viagem "0..*" o-- "1" Onibus : aloca >

' Onibus é composto por Poltronas (Composição: poltronas físicas pertencem ao ônibus)

Onibus "1" *-- "1..*" Poltrona : compõe >

' Passageiro realiza Compras

Passageiro "1" -- "0..*" CompraPassagem : efetua >

' Compra é composta por Bilhetes (Composição forte: o bilhete emitido não existe sem a compra)

CompraPassagem "1" *-- "1..*" BilhetePassagem : gera >

' Compra agrega Pagamento (Agregação: transação financeira complementa a compra)

CompraPassagem "1" o-- "1" Pagamento : liquidada por >

' Bilhete vincula Passageiro, Viagem e Poltrona

BilhetePassagem "0..*" -- "1" Viagem : refere-se a >

BilhetePassagem "1" -- "1" Poltrona : reserva assento >

BilhetePassagem "0..*" -- "1" Passageiro : emitido para >

' Reserva temporária vincula Poltrona e Viagem

ReservaPoltrona "0..1" -- "1" Poltrona : bloqueia >

ReservaPoltrona "0..*" -- "1" Viagem : na viagem >

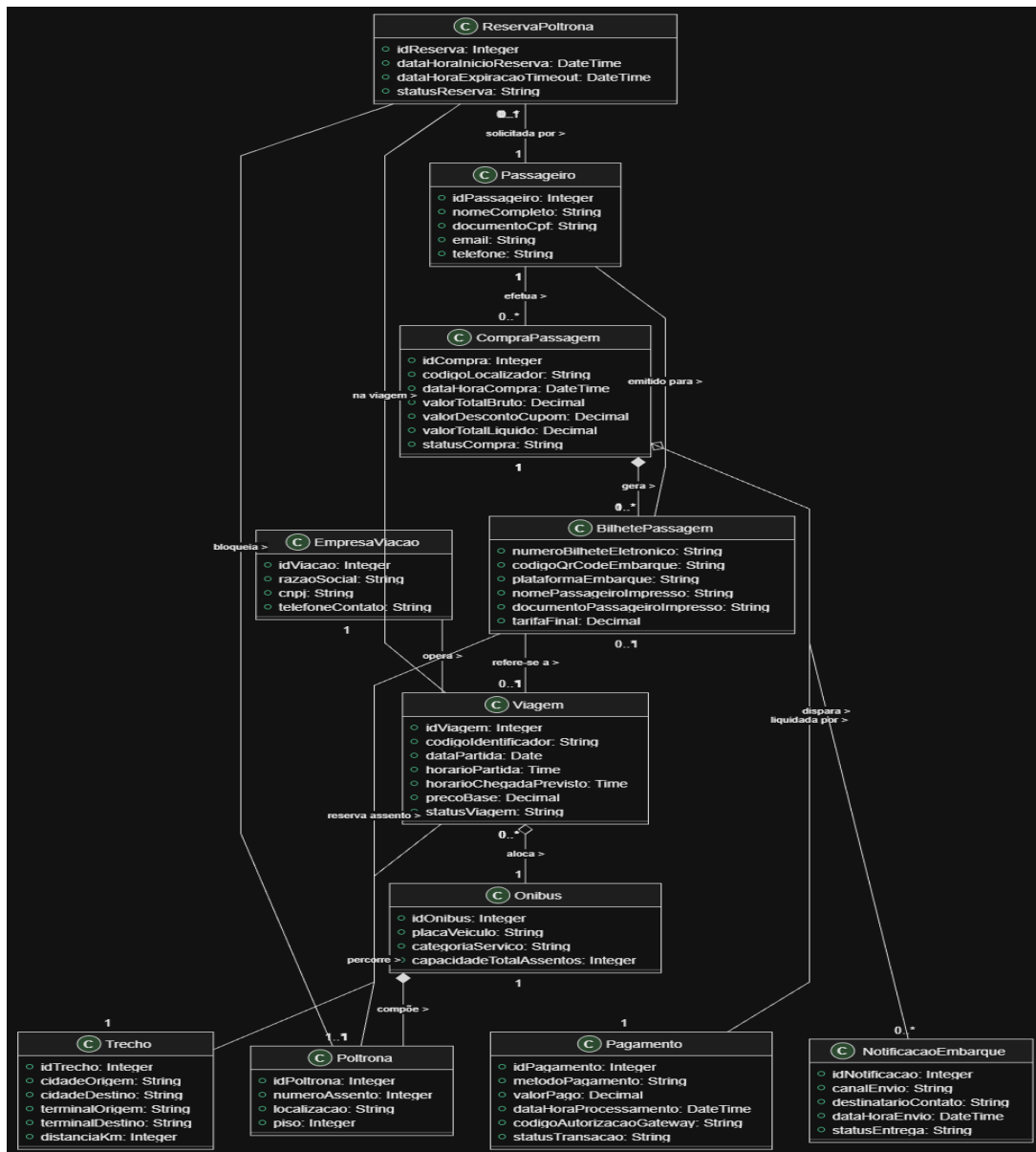
ReservaPoltrona "0..*" -- "1" Passageiro : solicitada por >

' Compra aciona Notificação

CompraPassagem "1" -- "0..*" NotificacaoEmbarque : dispara >

@endum1

Pós adicionar código UML ao diagrams.



Visualização organizada em fundo branco.

